Day-3
Python - Datatypes

What is a Data Type?

A **data type** tells Python **what kind of information** you are storing.

A data type defines the type of value a variable holds and determines how Python will process, store, and manipulate that value.

Names of Python Data Types

Text Type
- Str (String)

Numeric Types
- int, float, complex

Sequence Types
- list, tuple, range

Mapping Type
- dict

Set Types
- set, frozenset

Boolean Type
- bool

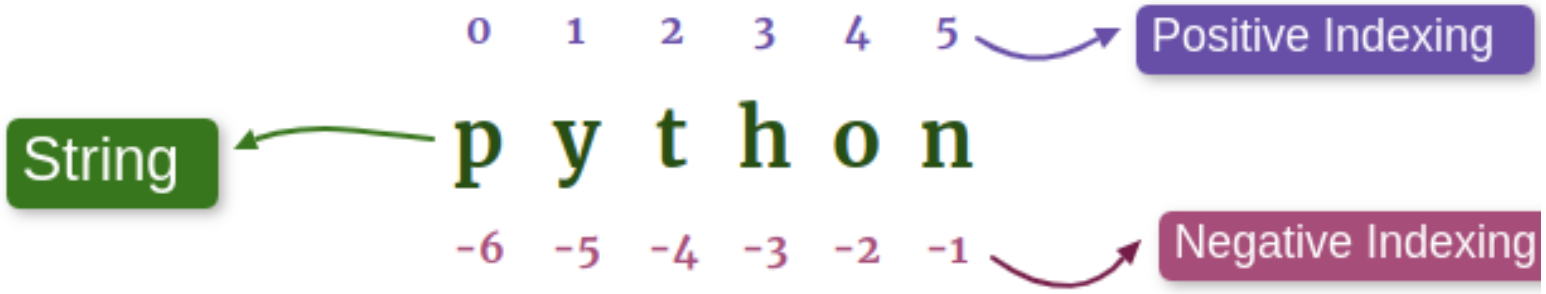
None Type
- None

Text Type: string

Description: String is used to represent textual data

Type: str

Example: A person's name: “ Rohit ”



Numeric Data type

Type: int

- Description: Stores whole numbers.
- Example: Number of students in a class = 25

Type: float

- Description: Stores decimal numbers.
- Example: Temperature = 37.5 °C

Type: complex

- Description: Number with real + imaginary part
- Example: Electrical circuit calculation, 3 + 4j

Sequence Types: list, tuple, range

SKILL COURSE

Ordered collections of items.

Type: list

Description: Ordered, changeable collection.

Example: Grocery list → ["milk", "bread", "eggs"]

Type: tuple

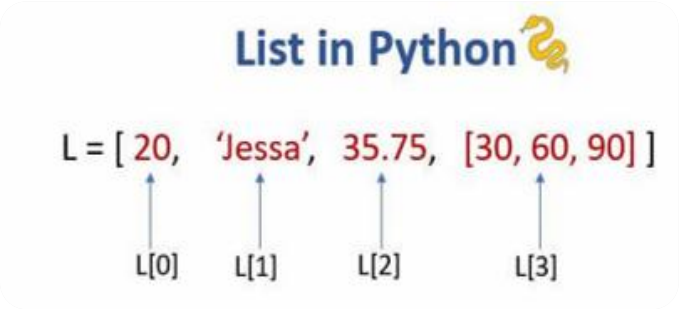
Description: Ordered, unchangeable collection.

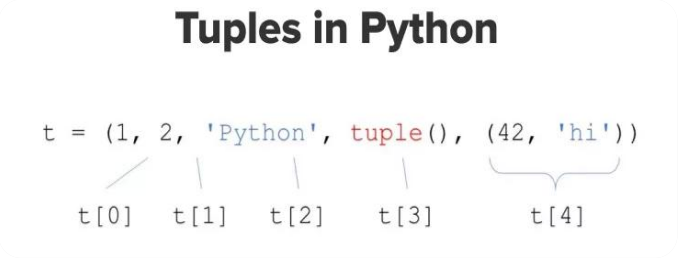
Example: Months of the year → ("Jan", "Feb", "Mar")

Type: range

Description: Sequence of numbers.

Example: Counting roll numbers → range(1, 51)





Python range(5)

0	1	2	3	4

Mapping Type: dict

Stores data in key–value pairs.
Keys are unique
Value can be any data type

```
country_capitals = {
  'Germany' : 'Berlin',   <----- element 1
  'Canada' : 'Ottawa',    <----- element 2
  'England' : 'London'    <----- element 3
}
```

key value

Description: Stores data in key–value pairs

Example:

{"name": "John", "age": 20}

Set Types: set, frozenset

- Unordered collections of unique items.
- No duplicate entries
- Order is not guaranteed

Type: set

- **Description:** Unordered collection of unique items.
- **Example:** {"apple", "banana", "orange"}

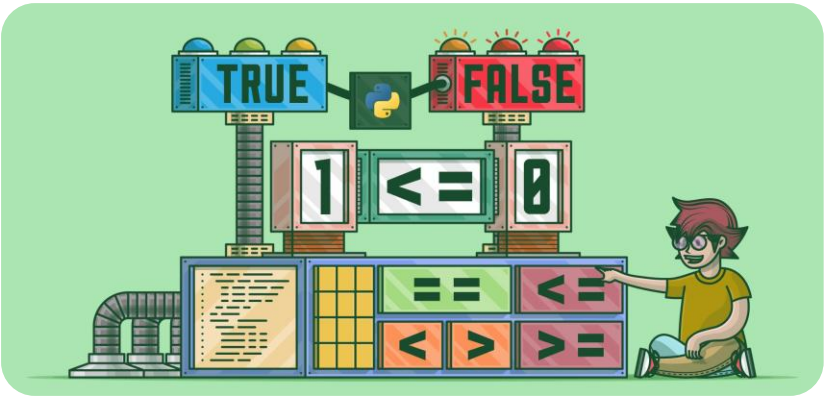
Type: frozenset

- **Description:** Same as set, but unchangeable.
- **Example:** frozenset(["a", "b", "c"])

SKILL COURSE

Boolean Type: bool

Represents True or False values.



Description: Represents True or False values.

Example: Is the door open? → True / False

None Type: None

Represents no value or empty state.

Example: x= None

Why Data Types Matter

Prevents errors: Python knows what operations are allowed

Efficiency: Correct type means efficient memory usage

Readability: Helps others understand what kind of data is expected

Debugging: Type errors are common

Real-Life Analogy: Like labeling jars — if you label “sugar” but you put salt, things go wrong.

Data Type	Mutable	Ordered	Allows Duplicates	Example
List	✓ Yes	✓ Yes	✓ Yes	`["A", "B"]`
Tuple	✗ No	✓ Yes	✓ Yes	`(1, 2, 3)`
Dict	✓ Yes	✗ No	Keys unique	`{"name": "Satish"}`
Set	✓ Yes	✗ No	✗ No	`{1, 2, 3}`
Range	✗ No	✓ Yes	✓ Yes	`range(1,10)`